

A2: Predicting Car Price

This notebook extends the from-scratch `LinearRegression` implementation from class with Xavier initialization, momentum, and stochastic/mini-batch/full-batch gradient descent, then uses it to predict used car selling prices.

What this notebook covers:

1. **Data preprocessing** — reusing the cleaning pipeline from A1
2. **Experimentation** — a full grid search across regression type (Normal/Ridge/Lasso/Polynomial), momentum, gradient descent method, weight initialization, and learning rate, tracked end-to-end with MLflow
3. **Best model training** — retraining the top configuration on the full dataset
4. **Evaluation** — test set performance and feature importance analysis
5. **Discussion** — key findings, including some unexpected results around training stability and gradient descent method choice

The corresponding web application (deployed as part of Task 3) is available in the `app/` folder, with deployment details in the repository's `README.md`.

GitHub: https://github.com/lmtiaz4201/car_price_predictor

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold
import pandas as pd
import re
import seaborn as sns
import itertools
import time
import sys
import os
import json
from IPython.display import display, Image, HTML
import warnings
import datetime
import mlflow
from sklearn.base import clone

import sys
import os
import json
import joblib

warnings.filterwarnings('ignore')
sys.path.append(os.path.abspath("."))
```

```

sns.set_style("whitegrid")
pd.set_option("display.float_format", lambda x: f"{x:.6f}")

# Eval Metrics
def mse(y_pred, y_true):
    s = np.size(y_true)
    z = ((y_pred - y_true) ** 2).sum() / s
    return z

def rmse(y_pred, y_true):
    mse = mse(y_pred, y_true)
    z = np.sqrt(mse)
    return z

def r2(y_pred, y_true):
    SST = np.sum((y_true - np.mean(y_true)) ** 2)
    SSE = np.sum((y_true - y_pred) ** 2)
    if SST == 0:
        return 0.0
    r2 = 1 - (SSE / SST)
    return r2

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder,
TargetEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.feature_selection import mutual_info_regression,
SelectKBest
from functools import partial

"""
Pipeline
Involved processes:
- SimpleImputer
- StandardScaler
- TargetEncoder
- OneHotEncoder
- KFold
- SelectKBest
- ColumnTransformer
"""

num_impute_pipeline = Pipeline(
    steps=[
        ("median_imput", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),

```

```

    ]
)
cat_impute_pipeline = Pipeline(
    steps=[
        ("cat_imput", SimpleImputer(strategy="most_frequent")),
        ("scale", StandardScaler()),
    ]
)
target_enc_pipeline = Pipeline(
    steps=[
        (
            "target_enc",
            TargetEncoder(target_type="continuous", smooth="auto",
random_state=42),
        ),
        ("scale", StandardScaler()),
    ]
)
scale_pipeline = Pipeline(steps=[("scale", StandardScaler())])
ct = ColumnTransformer(
    transformers=[
        ("num", num_impute_pipeline, ["mileage", "engine"]),
        ("cat", cat_impute_pipeline, ["seats"]),
        ("other_scale", scale_pipeline, ["year", "km_driven",
"max_power"]),
        (
            "OHE",
            OneHotEncoder(handle_unknown="ignore"),
            ["fuel", "seller_type", "transmission", "owner"],
        ),
        ("target_enc", target_enc_pipeline, ["brand"]),
    ],
    remainder="drop",
)

poly_pipeline = Pipeline(
    steps=[
        ("poly", PolynomialFeatures(degree=2, include_bias=False)),
        ("poly_scaler", StandardScaler()),
    ]
)

def sk(k=11):
    mi_fixed = partial(mutual_info_regression, random_state=42)
    sk_pipeline = Pipeline(steps=[("sk",
SelectKBest(score_func=mi_fixed, k=k))])
    return sk_pipeline

```

```

def feature_importance(coefs, feature_names=None, top_n=None):
    """
    Plots feature importance based on the magnitude of coefficients.

    IMPORTANT (per assignment note): this assumes input features are
    on the same scale (i.e. were scaled, e.g. with StandardScaler, before
    of fitting). Otherwise, coefficient magnitude is not a fair measure
    of importance.

    Parameters
    -----
    feature_names : list of str, optional
        Names of the features (excluding intercept). If None, generic
        names ('theta_1', 'theta_2', ...) are used.
    top_n : int, optional
        If given, only the top_n most important features are shown.
    """
    if feature_names is None:
        feature_names = [f"theta_{i+1}" for i in range(len(coefs))]

    assert len(feature_names) == len(
        coefs
    ), "feature_names length must match number of coefficients
    (excluding intercept)"

    importance = np.abs(coefs)
    order = np.argsort(importance)[::-1] # descending

    if top_n is not None:
        order = order[:top_n]

    sorted_names = [feature_names[i] for i in order]
    sorted_coefs = coefs[order]

    plt.figure(figsize=(8, max(4, 0.4 * len(sorted_names))))
    colors = ["#d62728" if c < 0 else "#1f77b4" for c in sorted_coefs]
    plt.barh(sorted_names, sorted_coefs, color=colors)
    plt.xlabel("Coefficient value")
    plt.title("Feature Importance (based on coefficient magnitude)")
    plt.gca().invert_yaxis()
    plt.axvline(0, color="black", linewidth=0.8)
    plt.tight_layout()
    plt.show()

def save_BT_results(model_params, final_train_mse_log,
                    final_train_r2_log, final_train_mse_exp, final_train_r2_exp, idx):
    res = {
        "model_info": model_params,
    }

```

```

        "mse_log": float(final_train_mse_log),
        "r2_log": float(final_train_r2_log),
        "mse_exp": float(final_train_mse_exp),
        "r2_exp": float(final_train_r2_exp),
        "datetime": datetime.datetime.now().isoformat(),
    }

    path = os.path.join(os.getcwd(), "saved_models")
    # create folder if not exists
    if not os.path.exists(path):
        os.makedirs(path)

    tr_file_path = os.path.join(path, "best_model_results"+str(idx)
+ ".json")

    if os.path.exists(tr_file_path):
        with open(tr_file_path, "r") as f:
            tr_file = json.load(f)
    else:
        tr_file = []

    tr_file.append(res)

    with open(tr_file_path, "w") as f:
        json.dump(tr_file, f, indent=4)

def predict(X, theta):
    """
    X -> (sample, features)
    theta -> (sample,)
    """
    y_hat = (X @ theta.reshape(-1,1)).flatten() # (batch,)
    return y_hat

class LinearRegression(object):
    def __init__(
        self,
        regularization,
        lr=0.001,
        method="batch",
        num_epochs=100,
        batch_size=16,
        n_splits=5,
        init_method="zeros",
        momentum=0.0,
        use_momentum=False,
        rng = None
    ):
        self.cv = KFold(n_splits, shuffle=True, random_state=42)

```

```

self.regularization = regularization
self.lr = lr
self.method = method
self.num_epochs = num_epochs
self.batch_size = batch_size
self.init_method = init_method
self.momentum = momentum
self.use_momentum = use_momentum
self.rng = rng
# Weight initialization
def _initialize_theta(self, n_features, m_samples):
    """
    n_features : number of columns in X, number of theata needed
    m_samples : number of samples in the current training fold
    """
    if self.init_method == "xavier":
        lower, upper = -(1.0 / np.sqrt(m_samples)), (1.0 /
np.sqrt(m_samples))
        theta = lower + self.rng.random(n_features) * (upper -
lower)
    else:
        theta = np.zeros(n_features)

    return theta

# Training
def fit(self, X_train, y_train, use_poly=False, k_features=11,
idx_=None):
    """
    X_train : shape -> (sample, features)
    y_train : shape -> (sample,)
    """
    self.kfold_scores = list()
    self.kfold_r2_scores = list()
    self.fold_pipelines = list()
    self.val_loss_old = np.inf

    for fold, (train_index, val_idx) in
enumerate(self.cv.split(X_train)):
        X_tr_raw = X_train.iloc[train_index]
        X_val_raw = X_train.iloc[val_idx]
        y_cross_train = y_train[train_index]
        y_cross_val = y_train[val_idx]

        ct_fold, sk_fold = clone(ct), sk(k_features)

        X_tr = ct_fold.fit_transform(X_tr_raw, y_cross_train)
        X_tr = sk_fold.fit_transform(X_tr, y_cross_train)

        X_val = ct_fold.transform(X_val_raw)

```

```

X_val = sk_fold.transform(X_val)

poly_fold = None
if use_poly:
    poly_fold = clone(poly_pipeline)
    X_tr = poly_fold.fit_transform(X_tr)
    X_val = poly_fold.transform(X_val)

# add intercept
X_cross_train = np.hstack([np.ones((X_tr.shape[0], 1)),
X_tr])
X_cross_val = np.hstack([np.ones((X_val.shape[0], 1)),
X_val])

self.fold_pipelines.append(
    {"ct": ct_fold, "sk": sk_fold, "poly": poly_fold}
)

# inti theta with zeros or xavier
self.theta = self._initialize_theta(
    n_features=X_cross_train.shape[1],
m_samples=X_cross_train.shape[0]
) # (F,)

# momentum book keeping
self.prev_step = np.zeros_like(self.theta) # (features,
1)

    with mlflow.start_run(run_name=f"Fold-{fold}",
nested=True) as fold_run:
        params = {
            "method": self.method,
            "lr": self.lr,
            "reg": type(self).__name__,
            "use_poly": use_poly,
            "init_method": self.init_method,
            "use_momentum": self.use_momentum,
            "momentum": self.momentum if self.use_momentum
else 0.0,
        }

        fold_dir = os.path.join(
            "saved_models", "fold_pipelines"+str(idx_),
fold_run.info.run_id
        )
        os.makedirs(fold_dir, exist_ok=True)
        joblib.dump(ct_fold, os.path.join(fold_dir, "ct.pkl"))
        joblib.dump(sk_fold, os.path.join(fold_dir, "sk.pkl"))
        if poly_fold is not None:
            joblib.dump(poly_fold, os.path.join(fold_dir,

```

```

"poly.pkl"))
    mlflow.log_artifacts(fold_dir,
artifact_path="preprocessors")
    mlflow.log_params(params=params)

    # training loops
    for epoch in range(self.num_epochs):
        # shuffle X_cross_train and y_cross_train
        idx = self.rng.permutation(X_cross_train.shape[0])
        X_cross_train = X_cross_train[idx]
        y_cross_train = y_cross_train[idx]

        if self.method == "sto":
            # stochastic
            for i in range(len(X_cross_train)):
                # get a single sample
                X_method_train =
X_cross_train[i].reshape(1, -1) # (S,F)
                y_method_train = y_cross_train[i] # (S,)
                # forward pass
                train_loss = self.train_model(
                    X_method_train, y_method_train
                )
            elif self.method == "mini":
                # mini batch
                for i in range(0, len(X_cross_train),
self.batch_size):

                    end_ = i + self.batch_size
                    X_method_train = X_cross_train[i:end_, :]
                    y_method_train = y_cross_train[i:end_]
                    train_loss = self.train_model(
                        X_method_train, y_method_train
                    )
                else:
                    # full batch
                    X_method_train = X_cross_train
                    y_method_train = y_cross_train
                    train_loss = self.train_model(X_method_train,
y_method_train)

            mlflow.log_metric(key="train_loss",
value=train_loss, step=epoch)

            # validation prediction
            # val mse
            y_val_hat_pred = self.predict(X_cross_val)
            val_loss_new = mse(y_val_hat_pred, y_cross_val)
            mlflow.log_metric(key="val_loss",
value=val_loss_new, step=epoch)

```

```

        # val r2
        val_r2_new = r2(y_val_hat_pred, y_cross_val)
        mlflow.log_metric(key="val_r2", value=val_r2_new,
step=epoch)

        if np.allclose(val_loss_new, self.val_loss_old):
            break
        self.val_loss_old = val_loss_new

        # show val loss after each epoch
        self.kfold_scores.append(val_loss_new)
        self.kfold_r2_scores.append(val_r2_new)
        print(f"Fold {fold}: {val_loss_new}")

def fit_final(
    self, X_train, y_train, save_weight=True, model_name=None,
    idx_=None, use_poly=False
):
    """
    Train the final model on the complete training dataset.

    X_train : shape -> (samples, features)
    y_train : shape -> (samples,)
    """
    # add intercept
    X_train = np.hstack([np.ones((X_train.shape[0], 1)), X_train])
    # Initialize theta
    self.theta = self._initialize_theta(
        n_features=X_train.shape[1], m_samples=X_train.shape[0]
    )

    # Momentum bookkeeping
    self.prev_step = np.zeros_like(self.theta)

    with mlflow.start_run(run_name="BEST_MODEL_TRAINING"):

        params = {
            "method": self.method,
            "lr": self.lr,
            "reg": type(self).__name__,
            "use_poly": use_poly,
            "init_method": self.init_method,
            "use_momentum": self.use_momentum,
            "momentum": self.momentum if self.use_momentum else
0.0,

            "num_epochs": self.num_epochs,
            "batch_size": self.batch_size,
        }

        mlflow.log_params(params)

```

```

# Training loop
for epoch in range(self.num_epochs):

    idx = self.rng.permutation(X_train.shape[0])
    X_train_batch = X_train[idx]
    y_train_batch = y_train[idx]

    if self.method == "sto":

        # Stochastic gradient descent
        for i in range(len(X_train_batch)):

            X_method_train = X_train_batch[i].reshape(1, -
1)
            y_method_train = y_train_batch[i]

            train_loss = self.train_model(
                X_method_train, y_method_train,
save_weight, idx_
            )

        elif self.method == "mini":

            # Mini-batch gradient descent
            for i in range(0, len(X_train_batch),
self.batch_size):

                end_ = i + self.batch_size

                X_method_train = X_train_batch[i:end_, :]
                y_method_train = y_train_batch[i:end_]

                train_loss = self.train_model(
                    X_method_train, y_method_train,
save_weight, idx_
                )

            else:

                # Full-batch gradient descent
                X_method_train = X_train_batch
                y_method_train = y_train_batch

                train_loss = self.train_model(
                    X_method_train, y_method_train, save_weight,
idx_
                )

    # Log training loss
    mlflow.log_metric("train_loss", train_loss,

```

```

step=epoch)

    # Final training prediction
    y_train_pred_log = self.predict(X_train)

    final_train_mse_log = mse(y_train_pred_log, y_train)
    final_train_r2_log = r2(y_train_pred_log, y_train)

    res = {"mse": final_train_mse_log, "r2":
final_train_r2_log}

    # Transform predictions and actual values back to original
price scale

    y_train_pred_exp = np.exp(y_train_pred_log)
    y_train_exp = np.exp(y_train)
    # Metrics in original price scale
    final_train_mse_exp = mse(y_train_pred_exp, y_train_exp)
    final_train_r2_exp = r2(y_train_pred_exp, y_train_exp)

    # Save both scale results into JSON
    model_key =
f"{model_name}_{self.use_momentum}_{self.momentum}_{self.method}_{self
.lr}"

    save_BT_results(
        model_key,
        float(final_train_mse_log),
        float(final_train_r2_log),
        float(final_train_mse_exp),
        float(final_train_r2_exp),
        idx=idx_,
    )

    mlflow.log_metric("final_train_mse_log",
final_train_mse_log)
    mlflow.log_metric("final_train_r2_log",
final_train_r2_log)

    print(f"Final training MSE log:
{final_train_mse_log:.4f}")
    print(f"Final training R2 log: {final_train_r2_log:.4f}")

    return res

def train_model(self, X, y, saved_weight=False, idx=None):
    """
    X/input : shape -> (batch, features)
    y/label : shape -> (batch,)
    """

```

```

y_hat = self.predict(X) # (batch,)
m = X.shape[0]

# loss = 0.5 * np.sum((y_hat - y) ** 2) # (1,1)

e = y_hat - y # (batch,)
grads = (1 / m) * X.T @ e + self.regularization.derivation(
    self.theta
) # (features,)

# update theta
step, self.theta = self.update_theta(
    self.lr, self.use_momentum, grads, self.theta,
self.momentum, self.prev_step
)

self.prev_step = step # (features, 1)

if saved_weight:
    path = os.path.join(os.getcwd(), "saved_models")
    os.makedirs(path, exist_ok=True)

    weight_file = os.path.join(path, "weights" + str(idx) +
".json")

    latest_weights = {
        "bias": float(self._bias()),
        "coefficients": self._coef().tolist(),
    }

    with open(weight_file, "w") as f:
        json.dump(latest_weights, f, indent=4)

    return mse(y_hat, y)

def update_theta(self, lr, use_momentum, grads, old_theta,
momentum, prv_step):
    step = lr * grads # (features, 1)
    if use_momentum:
        new_theta = old_theta - step + momentum * prv_step
    else:
        new_theta = old_theta - step

    return step, new_theta

def predict(self, X):
    # X's shape (batch, F)
    y_hat = (X @ self.theta.reshape(-1, 1)).flatten() # (batch,)
    return y_hat

```

```

def _coef(self):
    return self.theta[1:]

def _bias(self):
    return self.theta[0]

# Penalty classes
class LassoPenalty:
    def __init__(self, l):
        self.l = l

    def __call__(self, theta):
        return self.l * np.sum(np.abs(theta))

    def derivation(self, theta):
        return self.l * np.sign(theta)

class RidgePenalty:
    def __init__(self, l):
        self.l = l

    def __call__(self, theta):
        return self.l * np.sum(np.square(theta))

    def derivation(self, theta):
        return self.l * 2 * theta

class ElasticPenalty:
    def __init__(self, l=0.1, l_ratio=0.5):
        self.l = l
        self.l_ratio = l_ratio

    def __call__(self, theta):
        l1_contribution = self.l_ratio * self.l *
np.sum(np.abs(theta))
        l2_contribution = (1 - self.l_ratio) * self.l * 0.5 *
np.sum(np.square(theta))
        return l1_contribution + l2_contribution

    def derivation(self, theta):
        l1_derivation = self.l * self.l_ratio * np.sign(theta)
        l2_derivation = self.l * (1 - self.l_ratio) * theta
        return l1_derivation + l2_derivation

class NoPenalty:
    """
    'Normal' regression, i.e. no regularization at all.

```

Needed so that 'Normal' fits the same interface as Ridge/Lasso in Task 2's comparison loop (axis 1: polynomial, lasso, ridge, normal).

```
def __call__(self, theta):
    return 0

def derivation(self, theta):
    return np.zeros_like(theta)

class NormalLinearRegression(LinearRegression):
    def __init__(
        self,
        lr=0.001,
        method="batch",
        num_epochs=100,
        batch_size=16,
        n_splits=5,
        init_method="zeros",
        momentum=0,
        use_momentum=False,
        rng = None
    ):
        super().__init__(
            NoPenalty(),
            lr,
            method,
            num_epochs,
            batch_size,
            n_splits,
            init_method,
            momentum,
            use_momentum,
            rng
        )

class LassoLinearRegression(LinearRegression):
    def __init__(
        self,
        l=0.1,
        lr=0.001,
        method="batch",
        num_epochs=100,
        batch_size=16,
        n_splits=5,
        init_method="zeros",
        momentum=0,
        use_momentum=False,
```

```

        rng = None
    ):
        super().__init__(
            LassoPenalty(l),
            lr,
            method,
            num_epochs,
            batch_size,
            n_splits,
            init_method,
            momentum,
            use_momentum,
            rng
        )

class RidgeLinearRegression(LinearRegression):
    def __init__(
        self,
        l=0.1,
        lr=0.001,
        method="batch",
        num_epochs=100,
        batch_size=16,
        n_splits=5,
        init_method="zeros",
        momentum=0,
        use_momentum=False,
        rng = None
    ):
        super().__init__(
            RidgePenalty(l),
            lr,
            method,
            num_epochs,
            batch_size,
            n_splits,
            init_method,
            momentum,
            use_momentum,
            rng
        )

```

Load dataset

```

raw_data = pd.read_csv("Cars.csv")
raw_data.head()

```

	name	year	selling_price	km_driven
fuel \				
0	Maruti Swift Dzire VDI	2014	450000	145500
Diesel				
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000
Diesel				
2	Honda City 2017-2020 EXi	2006	158000	140000
Petrol				
3	Hyundai i20 Sportz Diesel	2010	225000	127000
Diesel				
4	Maruti Swift VXi BSIII	2007	130000	120000
Petrol				

	seller_type	transmission	owner	mileage	engine
max_power \					
0	Individual	Manual	First Owner	23.4 kmpl	1248 CC
bhp					
1	Individual	Manual	Second Owner	21.14 kmpl	1498 CC
bhp					
2	Individual	Manual	Third Owner	17.7 kmpl	1497 CC
bhp					
3	Individual	Manual	First Owner	23.0 kmpl	1396 CC
bhp					
4	Individual	Manual	First Owner	16.1 kmpl	1298 CC
bhp					

	torque	seats
0	190Nm@ 2000rpm	5.000000
1	250Nm@ 1500-2500rpm	5.000000
2	12.7@ 2,700(kgm@ rpm)	5.000000
3	22.4 kgm at 1750-2750rpm	5.000000
4	11.5@ 4,500(kgm@ rpm)	5.000000

Data pre-processing used in A1

```
X = raw_data.drop(['selling_price'], axis=1).copy()
y = raw_data['selling_price'].copy()

cat_cols = [i for i in X.columns if X[i].dtype == np.object_]
num_cols = [i for i in X.columns if X[i].dtype != np.object_]

# stripping extra spaces
X[cat_cols] = X[cat_cols].apply(lambda col: col.str.strip())
# removing string from mileage, engine, max_power
# pattern = re.compile(r"\d+(?:\.\d+)?")
# X["mileage"] = [float(re.findall(pattern, i)[0]) if pd.notna(i) else
# i for i in X['mileage'].values]
# X["engine"] = [float(re.findall(pattern, i)[0]) if pd.notna(i) else
```

```

i for i in X['engine'].values]
# X["max_power"] = X['max_power'].str.extract(r'(\d+(?:\.\d+)?)')[0].astype(float)

X["mileage"] = (
    X["mileage"]
    .astype(str)
    .str.extract(r'(\d+(?:\.\d+)?)')[0]
    .astype(float)
)

X["engine"] = (
    X["engine"]
    .astype(str)
    .str.extract(r'(\d+(?:\.\d+)?)')[0]
    .astype(float)
)

X["max_power"] = (
    X["max_power"]
    .astype(str)
    .str.extract(r'(\d+(?:\.\d+)?)')[0]
    .astype(float)
)

"""For the feature fuel, remove all rows with CNG and
LPG because CNG and LPG use a different
mileage system i.e., km/kg which is different
from kmfeaturepl for Diesel and Petrol"""

mask = (X['fuel'] == 'CNG') | (X['fuel'] == 'LPG')
index = X[mask].index
assert bool((y.iloc[index].index == X.iloc[index,:].index).all()), "x
and y index not same"

X = X.drop(index)
y = y.drop(index)

# taking brand name from name
X.loc[:, "name"] = [i[0] if len(i) > 1 else np.nan for i in
X['name'].str.split(' ')]
X.rename(columns={"name": "brand"}, inplace=True)

# Dropping torque column
X = X.drop("torque", axis=1)

# dropping rows related
mask = (X['fuel'] == 'CNG') | (X['fuel'] == 'LPG')
index = X[mask].index
assert bool((y.iloc[index].index == X.iloc[index,:].index).all()), "x
and y index not same"

```

```

X = X.drop(index)
y = y.drop(index)

# Dropping Test Drive Car as they have huge value and expensive
index = X[X['owner'] == 'Test Drive Car'].index
print(np.median(y[index]) > np.median(y.drop(index)))
X = X.drop(index)
y = y.drop(index)

True

# changing the data type
X[['mileage', 'engine', 'max_power']] = X[['mileage', 'engine',
'max_power']].astype(float)

num_cols = [i for i in X.columns if X[i].dtype is not np.object_]
cat_cols = [i for i in X.columns if X[i].dtype == np.object_]

# lower the text data to avoid uppercase and lowercase issue of same
data
X[cat_cols] = X[cat_cols].apply(lambda x:x.str.lower())

# check null values
X.isnull().sum()

brand          0
year           0
km_driven      0
fuel           0
seller_type    0
transmission   0
owner          0
mileage        214
engine         214
max_power      208
seats          214
dtype: int64

mask_all_null =
X[['mileage', 'engine', 'max_power', 'seats']].isna().all(axis=1)
indx = X[mask_all_null].index
X = X.drop(indx)
y = y.drop(indx)

X.isnull().sum()

brand          0
year           0
km_driven      0
fuel           0
seller_type    0

```

```

transmission    0
owner           0
mileage         6
engine          6
max_power       0
seats           6
dtype: int64

# total duplicates
print(X.duplicated().sum())
index = [*X[X.duplicated()]].index]
X = X.drop(index)
y = y.drop(index)

1675

# applying log to dependent variable, as
y_log = np.log(y)

```

Data split

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test, y_log_train, y_log_test =
train_test_split(
    X, y, y_log, test_size=0.20, random_state=42
)

```

Experimentation of Linear model with different setup

```

# fixed seed for Experimentation and Best model training
RNG = np.random.default_rng(42)

regression_types = ["normal", "lasso", "ridge", "polynomial"]
momentum_options = [False, True]
methods = ["sto", "mini", "batch"]
init_methods = ["zeros", "xavier"]
learning_rates = [0.01, 0.001, 0.0001]

db_path = os.path.join(
    os.getcwd(), "exps", "exp_train", "mlflow4.db"
)
tracking_uri = f"sqlite:/// {db_path.replace(os.sep, '/')}"
mlflow.set_tracking_uri(tracking_uri)
mlflow.set_experiment("A2-Car-Price-Experiment-v1")

```

```

combos = list(
    itertools.product(
        regression_types, momentum_options, methods, init_methods,
        learning_rates
    )
)
total_combos = len(combos)
print(f"Total combinations to run: {total_combos}")

results = []
start_time = time.time()

for i, (reg_type, use_momm, method, init_m, lr) in enumerate(combos,
start=1):
    use_poly = (reg_type == "polynomial")

    if reg_type == "lasso":
        ModelCls, kwargs = LassoLinearRegression, {"l": 0.1}
    elif reg_type == "ridge":
        ModelCls, kwargs = RidgeLinearRegression, {"l": 0.1}
    else:
        ModelCls, kwargs = NormalLinearRegression, {} # covers
"normal" and "polynomial"

    run_name = f"{reg_type}_mom{use_momm}_{method}_{init_m}_lr{lr}"

    combo_start = time.time()
    print(f"[{i}/{total_combos}] Running: {run_name} ...", end=" ",
flush=True)

    with mlflow.start_run(run_name=run_name) as parent_run:
        mlflow.log_params(
            {
                "regression_type": reg_type,
                "use_momentum": use_momm,
                "method": method,
                "init_method": init_m,
                "learning_rate": lr,
            }
        )

        model = ModelCls(
            lr=lr,
            method=method,
            num_epochs=30,
            batch_size=64,
            n_splits=5,
            init_method=init_m,
            momentum=0.9 if use_momm else 0.0,
            use_momentum=use_momm,

```

```

        rng=RNG,
        **kwargs,
    )
    model.fit(X_train, y_log_train.values, use_poly=use_poly,
idx_=4)

    avg_val_mse = np.mean(model.kfold_scores)
    avg_val_r2 = np.mean(model.kfold_r2_scores)
    mlflow.log_metric("avg_kfold_val_mse", avg_val_mse)

    results.append(
        {
            "regression_type": reg_type,
            "momentum": use_momm,
            "method": method,
            "init": init_m,
            "lr": lr,
            "avg_val_mse": avg_val_mse,
            "avg_val_r2": avg_val_r2,
            "mlflow_run_id": parent_run.info.run_id
        }
    )

    combo_time = time.time() - combo_start
    elapsed = time.time() - start_time
    avg_per_combo = elapsed / i
    remaining = avg_per_combo * (total_combos - i)
    print(
        f"done in {combo_time:.1f}s | avg_val_mse={avg_val_mse:.4f} |
"
        f"elapsed={elapsed/60:.1f}min | est.
remaining={remaining/60:.1f}min"
    )

    results_df = pd.DataFrame(results)

    results_df_sorted = results_df.sort_values(by="avg_val_mse",
ascending=True)
    results_df_sorted.head(10)

    res_save_folder = os.path.join(os.getcwd(), "results_folder")
    os.makedirs(res_save_folder, exist_ok=True)
    results_df.to_csv(os.path.join(res_save_folder, 'results4.csv'))

# to retrieve best model's saved transformation .pkl for further use
    from mlflow.tracking import MlflowClient

    db_path = os.path.join(
        os.getcwd(), "exps", "exp_train", "mlflow4.db"
    )

```

```

tracking_uri = f"sqlite:///{{db_path.replace(os.sep, '/')}}"
mlflow.set_tracking_uri(tracking_uri)
client = MlflowClient()

base_dir = os.path.join("saved_models", "fold_pipelines4")
folder_ids = os.listdir(base_dir)

rows = []
for run_id in folder_ids:
    try:
        fold_run = client.get_run(run_id)
    except mlflow.exceptions.MlflowException:
        rows.append({"folder": run_id, "status": "no matching run in
mlflow.db"})
        continue

    parent_id = fold_run.data.tags.get("mlflow.parentRunId")
    parent = client.get_run(parent_id) if parent_id else None

    rows.append({
        "folder": run_id,
        "fold_run_name": fold_run.info.run_name,
        "fold_index": fold_run.data.params.get("fold"),
        "parent_run_id": parent_id,
        "combo_name": parent.info.run_name if parent else None,
        "regression_type": parent.data.params.get("regression_type")
if parent else None,
        "method": parent.data.params.get("method") if parent else
None,
        "learning_rate": parent.data.params.get("learning_rate") if
parent else None,
        "init_method": parent.data.params.get("init_method") if parent
else None,
        "use_momentum": parent.data.params.get("use_momentum") if
parent else None
    })

mapping_df = pd.DataFrame(rows)
merged = results_df.merge(
    mapping_df, left_on='mlflow_run_id', right_on='parent_run_id',
    how='left'
)
best_combo_folds = merged[merged['mlflow_run_id'] ==
results_df.sort_values('avg_val_mse').iloc[0]['mlflow_run_id']]
merged.to_csv(os.path.join(res_save_folder, 'merged4.csv'))
best_combo_folds.to_csv(os.path.join(res_save_folder,
'best_combo_folds4.csv'))

```

Experiments evaluation

```
results_df = pd.read_csv(r'results_folder/results4.csv')
results_df = results_df.drop("Unnamed: 0",axis=1)
```

```
merged_df = pd.read_csv(r'results_folder/merged4.csv')
merged_df = merged_df.drop("Unnamed: 0",axis=1)
```

```
top_10_exp = results_df.sort_values(by = "avg_val_mse",
ascending=True).head(10)
top_10_exp
```

	regression_type	momentum	method	init	lr	avg_val_mse	\
113	polynomial	False	sto	xavier	0.000100	0.076117	
110	polynomial	False	sto	zeros	0.000100	0.076205	
130	polynomial	True	sto	xavier	0.001000	0.076236	
127	polynomial	True	sto	zeros	0.001000	0.076260	
117	polynomial	False	mini	xavier	0.010000	0.076390	
114	polynomial	False	mini	zeros	0.010000	0.076880	
21	normal	True	sto	xavier	0.010000	0.085516	
1	normal	False	sto	zeros	0.001000	0.086216	
18	normal	True	sto	zeros	0.010000	0.086721	
4	normal	False	sto	xavier	0.001000	0.086795	

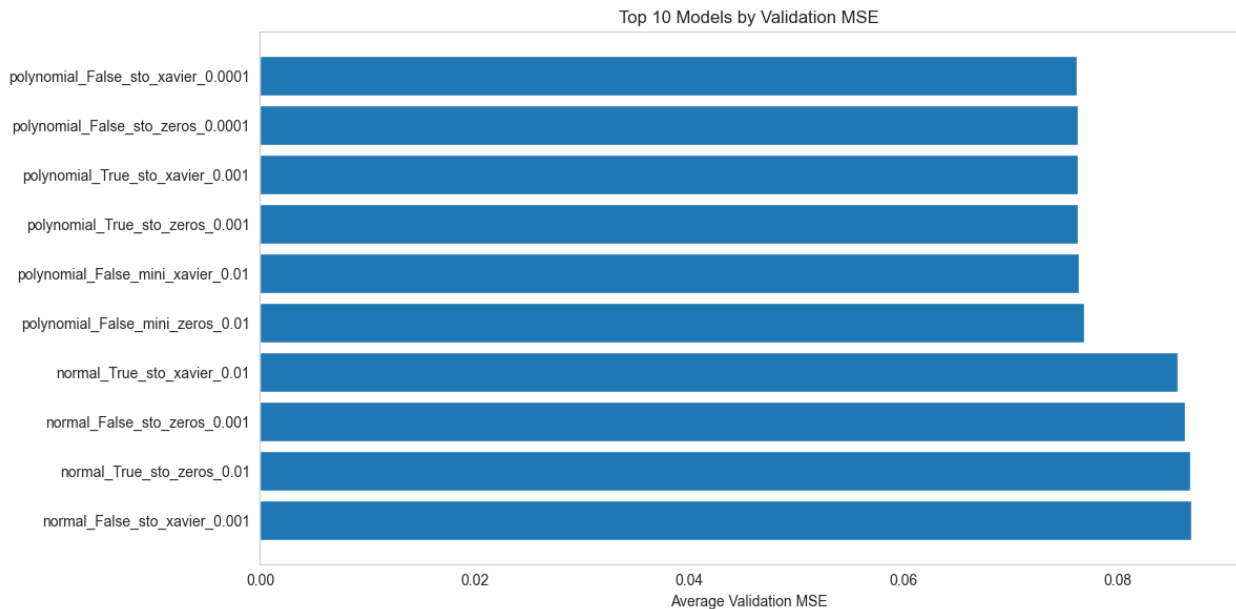
	avg_val_r2	mlflow_run_id
113	0.868088	7056641acfb42688c9fec3e5b29c5cc
110	0.867929	7bfc2d36e6e04133ab80c58a387bf977
130	0.867884	a9d27a46406e4c2bac26d46d17756a32
127	0.867834	6f51d7fd37b64bd9baa112b84c1c92ce
117	0.867619	601f823ecb7f449d82b5a22f08aff8ed
114	0.866769	7aa2e1b2c11744688f6e56d8f53bf576
21	0.851588	915d750b410f47f4b2b3f3fdee813c43
1	0.850427	ac297b6535ba4d5fa8a001f3f1c6f7c6
18	0.849562	ff2d832a2ece4ed0a30df0ee9ea410a1
4	0.849386	ee483f86d7214c62bca70207a029eec6

```
top10 = top_10_exp.iloc[:,:]
top10["model"] = (
    top10["regression_type"] + "_" +
    top10["momentum"].astype(str) + "_" +
    top10["method"] + "_" +
    top10["init"] + "_" +
    top10["lr"].astype(str)
)
```

Top 10 Models by Validation MSE

```
temp = top10.sort_values('avg_val_mse', ascending=False)
plt.figure(figsize=(12, 6))
plt.barh(temp["model"], temp["avg_val_mse"])
```

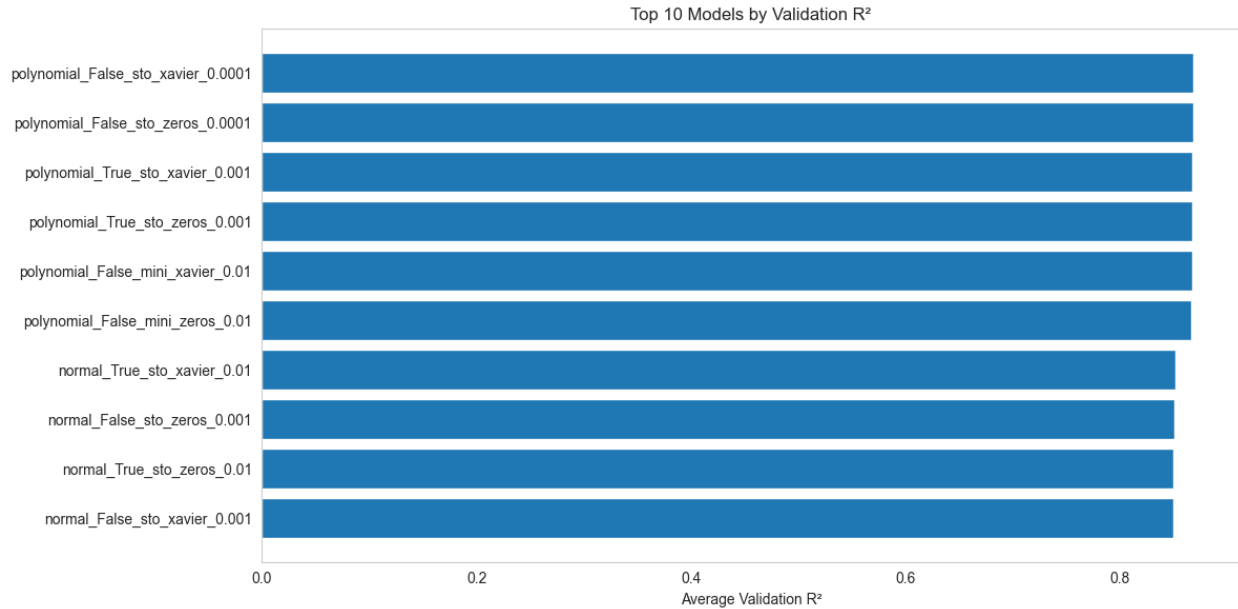
```
plt.xlabel("Average Validation MSE")
plt.title("Top 10 Models by Validation MSE")
plt.tight_layout()
plt.grid(False)
plt.show()
```



The best configuration is **polynomial_False_sto_xavier_0.0001**, closely followed by five other polynomial setups using mini-batch or stochastic gradient descent. Normal regression models fill the remaining top ten spots (**normal_False_sto_xavier_0.001** to **normal_True_sto_zeros_0.01**). Polynomial models consistently outperformed normal models, while momentum and initialization methods showed negligible performance differences.

Top 10 Models by Validation R^2

```
top10_r2 = top10.sort_values("avg_val_r2", ascending=False).head(10)
temp1 = top10_r2.sort_values("avg_val_r2", ascending=True)
plt.figure(figsize=(12, 6))
plt.barh(temp1["model"], temp1["avg_val_r2"])
plt.xlabel("Average Validation  $R^2$ ")
plt.title("Top 10 Models by Validation  $R^2$ ")
plt.tight_layout()
plt.grid(False)
plt.show()
```



The top configuration by Average Validation R^2 is **polynomial_False_sto_xavier_0.0001**, followed by five other polynomial setups using mini-batch or stochastic descent. Normal regression models fill the bottom four spots (**normal_False_sto_xavier_0.001** through **normal_True_sto_zeros_0.01**). Polynomial models consistently achieved higher R^2 scores, while momentum and initialization options showed minimal impact.

Divergence Rate by Learning Rate

```
from mlflow.tracking import MlflowClient
db_path = os.path.join(
    os.getcwd(), "exps", "exp_train", "mlflow4.db"
)
tracking_uri = f"sqlite:///{"db_path".replace(os.sep, '/')}
mlflow.set_tracking_uri(tracking_uri)
client = MlflowClient()

experiment_id = "1"
runs = mlflow.search_runs(experiment_ids=[experiment_id])
parents = runs[runs["tags.mlflow.parentRunId"].isna()].copy()
children = runs[runs["tags.mlflow.parentRunId"].notna()].copy()

children["diverged"] = (
    (children["metrics.val_loss"] > 1e10) |
    children["metrics.val_loss"].isna() |
    np.isinf(children["metrics.val_loss"])
)

divergence_by_lr = children.groupby("params.lr")
["diverged"].agg(["sum", "count"])
divergence_by_lr["rate"] = divergence_by_lr["sum"] /
```

```

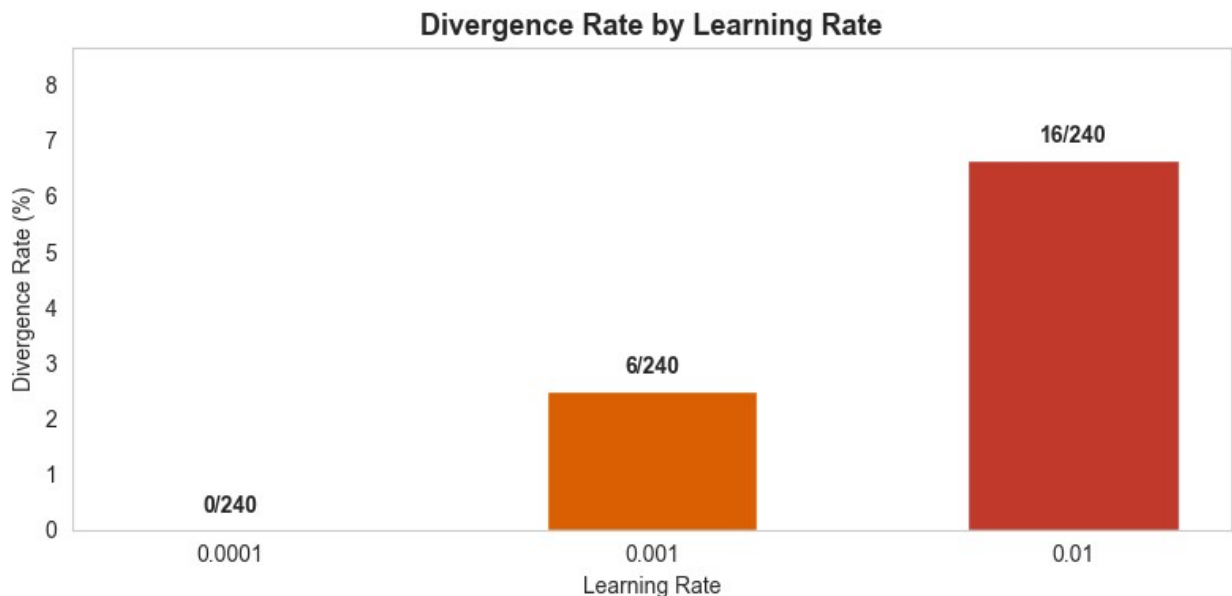
divergence_by_lr["count"]

fig, ax = plt.subplots(figsize=(8,4))
bars = ax.bar(
    divergence_by_lr.index.astype(str),
    divergence_by_lr["rate"] * 100,
    color=["#2b5c8f", "#d95f02", "#c0392b"],
    width=0.5,
)

for bar, (idx, row) in zip(bars, divergence_by_lr.iterrows()):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.3,
        f"{int(row['sum'])}/{int(row['count'])}",
        ha="center", fontsize=10, fontweight="bold"
    )

ax.set_xlabel("Learning Rate")
ax.set_ylabel("Divergence Rate (%)")
ax.set_title("Divergence Rate by Learning Rate", fontweight="bold",
    fontsize=13)
ax.set_ylim(0, max(divergence_by_lr["rate"] * 100) * 1.3)
plt.tight_layout()
plt.grid(False)
plt.show()

```



Higher learning rates cause more training failures. At 0.0001, no runs diverged. At 0.001, 6 out of 240 failed. At 0.01, 16 out of 240 failed — over 6%. Larger learning rates make weight updates too big, causing the model to overshoot and blow up.

MLflow Run List (sorted by avg_kfold_val_mse)

```
display(Image(filename=r'images/A4.png', height=1000, width=1000))
```

				Metrics		Parameters			
	Run Name	Created	Duration	avg_kfold_val_mse	init_method	learning_rate	method	regression_type	use_momentum
	Additional runs 100	-		31.6357...	-	0.004564	-	-	-
	polynomial_momFalse_...	18 hours ago	27.6s	0.076116...	xavier	0.0001	sto	polynomial	False
	polynomial_momFalse_...	18 hours ago	32.3s	0.076204...	zeros	0.0001	sto	polynomial	False
	polynomial_momTrue_s...	18 hours ago	32.8s	0.076235...	xavier	0.001	sto	polynomial	True
	polynomial_momTrue_s...	18 hours ago	32.9s	0.076259...	zeros	0.001	sto	polynomial	True
	polynomial_momFalse_...	18 hours ago	11.2s	0.076389...	xavier	0.01	mini	polynomial	False
	polynomial_momFalse_...	18 hours ago	8.0s	0.076880...	zeros	0.01	mini	polynomial	False
	normal_momTrue_sto_x...	19 hours ago	34.8s	0.085516...	xavier	0.01	sto	normal	True
	normal_momFalse_sto_...	19 hours ago	36.3s	0.086216...	zeros	0.001	sto	normal	False
	normal_momTrue_sto_z...	19 hours ago	32.9s	0.086720...	zeros	0.01	sto	normal	True
	normal_momFalse_sto_...	19 hours ago	25.8s	0.086794...	xavier	0.001	sto	normal	False
	normal_momFalse_sto_...	19 hours ago	26.7s	0.096278...	xavier	0.01	sto	normal	False
	lasso_momFalse_sto_xa...	19 hours ago	26.6s	0.124411...	xavier	0.01	sto	lasso	False
	lasso_momFalse_sto_z...	19 hours ago	29.4s	0.124977...	zeros	0.01	sto	lasso	False
	lasso_momFalse_sto_z...	19 hours ago	27.4s	0.142568...	zeros	0.001	sto	lasso	False

This table ranks all 144 experiments by validation MSE, best first. Polynomial regression with stochastic gradient descent dominates the top, showing lower error and outperforming normal, lasso, and ridge configurations.

Effect of Regularization Type on Validation MSE

```
import matplotlib.image as mpimg
img1 = mpimg.imread("images/less than 0.2.png")
```

```

img2 = mpimg.imread("images/less than 2.png")

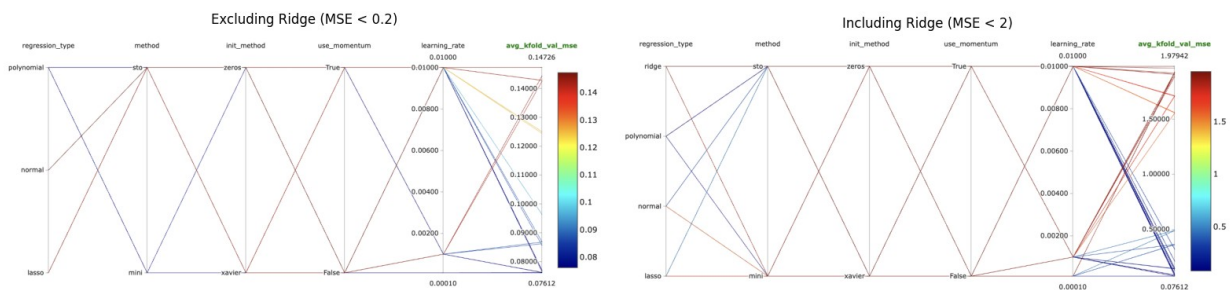
fig, axes = plt.subplots(1, 2, figsize=(16,16))

axes[0].imshow(img1)
axes[0].axis("off")
axes[0].set_title("Excluding Ridge (MSE < 0.2)")

axes[1].imshow(img2)
axes[1].axis("off")
axes[1].set_title("Including Ridge (MSE < 2)")

plt.tight_layout()
plt.show()

```



These two parallel coordinates plots reveal a key finding: Ridge regularization performs dramatically worse than the other regression types. Image 1 (MSE < 0.2, excluding Ridge) shows Polynomial, Normal, and Lasso clustered tightly between 0.076–0.15, with learning rate as the dominant driver — $lr=0.0001$ consistently produces the best (dark blue) results, while $lr=0.01$ produces the worst (red). Image 2 (MSE < 2, including Ridge) reveals Ridge configurations reaching MSE values up to ~1.98, over 10x worse than any other type. This suggests Ridge's default regularization strength over-penalized coefficients, substantially degrading predictive performance compared to Lasso and unregularized alternatives.

Validation R^2 by Method

```

converged = children[
    ~children["diverged"] &
    children["metrics.val_r2"].notna() &
    (children["metrics.val_r2"] > -1000)
]

# summary table from the filtered (catastrophic-outlier-excluded) data
summary = converged.groupby("params.method")["metrics.val_r2"].agg(
    count="count",
    mean="mean",
    median="median",
    min="min",
    max="max",

```

```

        std="std"
    ).round(3)

summary = summary.reindex(["batch", "mini", "sto"]) # logical order
summary.columns = ["Count", "Mean R2", "Median R2", "Min R2", "Max
R2", "Std Dev"]

fig, ax = plt.subplots(figsize=(9, 2.8))
ax.axis("off")

col_labels = ["Method", "Count", "Mean R2", "Median R2", "Min R2",
"Max R2", "Std Dev"]
row_data = [
    ["Batch", "240", f"{summary.loc['batch', 'Mean R2']:.2f}",
f"{summary.loc['batch', 'Median R2']:.2f}",
    f"{summary.loc['batch', 'Min R2']:.2f}", f"{summary.loc['batch',
'Max R2']:.2f}", f"{summary.loc['batch', 'Std Dev']:.2f}"],
    ["Mini-batch", "240", f"{summary.loc['mini', 'Mean R2']:.2f}",
f"{summary.loc['mini', 'Median R2']:.2f}",
    f"{summary.loc['mini', 'Min R2']:.2f}", f"{summary.loc['mini',
'Max R2']:.2f}", f"{summary.loc['mini', 'Std Dev']:.2f}"],
    ["Stochastic", str(summary.loc['sto', 'Count']),
f"{summary.loc['sto', 'Mean R2']:.2f}", f"{summary.loc['sto', 'Median
R2']:.2f}",
    f"{summary.loc['sto', 'Min R2']:.2f}", f"{summary.loc['sto', 'Max
R2']:.2f}", f"{summary.loc['sto', 'Std Dev']:.2f}"],
]

table = ax.table(cellText=row_data, colLabels=col_labels,
cellloc="center", loc="center", bbox=[0, 0, 1, 1])
table.auto_set_font_size(False)
table.set_fontsize(10.5)

header_color = "#2b5c8f"
row_colors = ["#f2f2f2", "#ffffff"]

for (row, col), cell in table.get_celld().items():
    cell.set_edgecolor("#d9d9d9")
    if row == 0:
        cell.set_facecolor(header_color)
        cell.set_text_props(color="white", fontweight="bold")
        cell.set_height(0.2)
    else:
        cell.set_facecolor(row_colors[(row - 1) % 2])
        cell.set_height(0.2)
        if col == 0:
            cell.set_text_props(fontweight="bold", ha="left")
        else:
            cell.set_text_props(fontfamily="monospace")

```

```
ax.set_title("Validation R2 by Method (Catastrophic Divergences Excluded)", fontweight="bold", fontsize=13, pad=15)
plt.tight_layout()
plt.show()
```

Validation R² by Method (Catastrophic Divergences Excluded)

Method	Count	Mean R ²	Median R ²	Min R ²	Max R ²	Std Dev
Batch	240	-240.04	-272.59	-307.35	-54.71	73.41
Mini-batch	240	-90.11	-54.33	-295.69	0.88	100.94
Stochastic	210	-2.02	0.30	-28.14	0.88	5.78

Full-batch gradient descent failed to achieve positive validation R² in any of its 240 fold-runs (mean -240.04, range -307.35 to -54.71), and mini-batch performed poorly in the majority of cases (median R² = -54.33), while stochastic gradient descent was the only method that reliably reached strong fits (median R² = 0.30, max 0.88). This is consistent with our earlier finding that update frequency per epoch is a major factor in this experiment: with a fixed epoch budget, full-batch gradient descent receives only one parameter update per epoch, compared to potentially thousands for stochastic descent, leaving it unable to converge meaningfully within the same training budget.

Train best model

```
# load saved transformations from .pkl files
bestModelMLflowId = top_10_exp.iloc[0]['mlflow_run_id']
best_fold_folder_name = merged_df[merged_df['mlflow_run_id'] ==
bestModelMLflowId].sort_values(by="avg_val_mse", ascending=True)
n = best_fold_folder_name.iloc[0]['folder']
path = os.path.join(os.getcwd(), "saved_models", "fold_pipelines4", n)
val = [os.path.join(path,i) for i in os.listdir(path)]
keys = [i.split(os.sep)[-1].split('.')[0] for i in val]
trans_data = {k:v for k,v in zip(keys,val)}
trans_data

{'sk': '/Users/imtiaz/Documents/mlass2/mlass2n/A2 - Predicting Car
Prices/saved_models/fold_pipelines4/81b84d6e04a24f1c86e365457f023bd1/
sk.pkl',
 'poly': '/Users/imtiaz/Documents/mlass2/mlass2n/A2 - Predicting Car
Prices/saved_models/fold_pipelines4/81b84d6e04a24f1c86e365457f023bd1/
poly.pkl',
 'ct': '/Users/imtiaz/Documents/mlass2/mlass2n/A2 - Predicting Car
```

```
Prices/saved_models/fold_pipelines4/81b84d6e04a24f1c86e365457f023bd1/
ct.pkl'}
```

```
best_row = best_fold_folder_name.iloc[0]
print("Best combo:", best_row.to_dict())
```

```
# load transformers
```

```
ct_trans = joblib.load(trans_data["ct"])
sk_trans = joblib.load(trans_data["sk"])
poly_trans = joblib.load(trans_data["poly"])
```

```
X_train_tr = ct_trans.transform(X_train)
X_train_tr = sk_trans.transform(X_train_tr)
```

```
X_train_tr_poly = poly_trans.transform(X_train_tr)
```

```
Best combo: {'regression_type_x': 'polynomial', 'momentum': False,
'method_x': 'sto', 'init': 'xavier', 'lr': 0.0001, 'avg_val_mse':
0.076116553310284, 'avg_val_r2': 0.868088499815028, 'mlflow_run_id':
'7056641acfb42688c9fec3e5b29c5cc', 'folder':
'81b84d6e04a24f1c86e365457f023bd1', 'fold_run_name': 'Fold-3',
'fold_index': nan, 'parent_run_id':
'7056641acfb42688c9fec3e5b29c5cc', 'combo_name':
'polynomial_momFalse_sto_xavier_lr0.0001', 'regression_type_y':
'polynomial', 'method_y': 'sto', 'learning_rate': 0.0001,
'init_method': 'xavier', 'use_momentum': False}
```

```
db_path = os.path.join(os.getcwd(), "exps", "best_model_train",
"mlflow4.db")
```

```
tracking_uri = f"sqlite:/// {db_path.replace(os.sep, '/')}"
```

```
mlflow.set_tracking_uri(tracking_uri)
```

```
mlflow.set_experiment("A2-Car-Price-Experiment-v1")
```

```
ModelCls_map = {
    "normal": NormalLinearRegression,
    "polynomial": NormalLinearRegression,
    "lasso": LassoLinearRegression,
    "ridge": RidgeLinearRegression,
}
```

```
X_train_data_map = {
    "normal": X_train_tr,
    "lasso": X_train_tr,
    "ridge": X_train_tr,
    "polynomial": X_train_tr_poly,
}
```

```
kwargs_map = {"normal": {}, "polynomial": {}, "lasso": {"l": 0.1},
"ridge": {"l": 0.1}}
```

```
reg_type = best_row["regression_type_x"]
```

```
up = reg_type == "polynomial"
```

```

best_model = ModelCls_map[reg_type](
    lr=best_row["lr"],
    method=best_row["method_x"],
    num_epochs=30, # same as experiment
    batch_size=64, # same as experiment
    n_splits=5, # same as experiment
    init_method=best_row["init"],
    momentum=0.9 if best_row["momentum"] else 0.0,
    use_momentum=bool(best_row["momentum"]),
    rng = RNG,
    **kwargs_map[reg_type],
)
results = best_model.fit_final(
    X_train_data_map[reg_type],
    y_log_train.values,
    save_weight=True,
    model_name=reg_type,
    idx_=4,
    use_poly=up
)

```

Best Model train Evaluation: MSE, RMSE and R² across Scales

```

with open(r"saved_models/best_model_results4.json", "r") as f:
    metrics_data = json.load(f)[0]

mse_log = metrics_data["mse_log"]
mse_exp = metrics_data["mse_exp"]
r2_log = metrics_data["r2_log"]
r2_exp = metrics_data["r2_exp"]

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5),
    gridspec_kw={"width_ratios": [1.1, 1]})

ax_table = axes[0]
ax_table.axis("off")

col_labels = ["Metric", "Log Scale", "Original Scale (Exp)"]
row_data = [
    ["MSE", f"{mse_log}", f"{mse_exp}"],
    ["RMSE", f"{np.sqrt(mse_log)}", f"{np.sqrt(mse_exp)}"],
    ["R2", f"{r2_log}", f"{r2_exp}"],
]

table = ax_table.table(
    cellText=row_data,
    colLabels=col_labels,
    cellLoc="center",
    loc="center",

```

```

        bbox=[0, 0, 1, 1],
    )

table.auto_set_font_size(False)
table.set_fontsize(11)

header_color = "#2b5c8f"
row_colors = ["#f2f2f2", "#ffffff"]

for (row, col), cell in table.get_celld().items():
    cell.set_edgecolor("#d9d9d9")
    if row == 0:
        cell.set_facecolor(header_color)
        cell.set_text_props(color="white", fontweight="bold")
        cell.set_height(0.18)
    else:
        cell.set_facecolor(row_colors[(row - 1) % 2])
        cell.set_height(0.18)
        if col == 0:
            cell.set_text_props(fontweight="bold", ha="left")
        else:
            cell.set_text_props(fontfamily="monospace")

ax_table.set_title("Model Evaluation Summary", fontweight="bold",
fontsize=13, pad=15)

categories = ["Log Scale", "Exp Scale"]
r2_values = [r2_log, r2_exp]

bars_r2 = axes[1].bar(categories, r2_values, color="#d95f02",
width=0.5)
axes[1].bar_label(bars_r2, fmt="%.4f", padding=3)
axes[1].set_ylim(0, max(r2_values) * 1.2 if max(r2_values) > 0 else
1.0)
axes[1].set_title("R2 Score", fontweight="bold")
axes[1].set_ylabel("R2 Score")

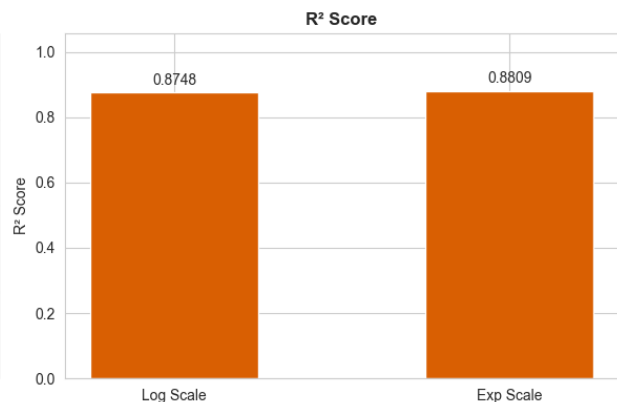
fig.suptitle("Best Model train Evaluation: MSE, RMSE and R2 across
Scales", fontsize=13)
plt.tight_layout()
plt.show()

```

Best Model train Evaluation: MSE, RMSE and R² across Scales

Model Evaluation Summary

Metric	Log Scale	Original Scale (Exp)
MSE	0.07224817269308736	31913771914.925167
RMSE	0.26879020200350934	178644.26079481302
R ²	0.8748225838433509	0.8808596875400956



The final model (best hyperparameter combination from the grid search) was trained on the full training set and evaluated on both the log-transformed target and the original price scale. It achieved $R^2 = 0.8748$ on the log scale and $R^2 = 0.8809$ on the original price scale, indicating strong, consistent fit across both representations. MSE and RMSE appear large in absolute terms on the original scale (RMSE $\approx 178,644$) as the car prices are large-magnitude values.

Test best model on test set

```
# using X_test_poly as our best model is polynomial
X_test_tr = ct_trans.transform(X_test)
X_test_tr = sk_trans.transform(X_test_tr)
X_test_poly = poly_trans.transform(X_test_tr)
X_test_poly = np.hstack([np.ones((X_test_poly.shape[0], 1)),
X_test_poly])

y_pred_exp = np.exp(best_model.predict(X_test_poly))
y_pred_exp

# load best model's weight
def predict_(X):
    try:
        path = os.path.join(os.getcwd(), "saved_models")
        tr_file_path = os.path.join(path, "weights4.json")
        if os.path.exists(tr_file_path):
            with open(tr_file_path, "r") as f:
                tr_file = json.load(f) # load the last bias and coef

            bias, coef = tr_file.values() # (batch,)
            theta = np.array([bias] + coef)
            y_hat_log = predict(X, theta)
            y_hat_exp = np.exp(y_hat_log)
            return y_hat_exp, y_hat_log
```

```

        else:
            print("file does not exists")

    except Exception as e:
        print(e)

y_pred_cust_exp, y_pred_cust_log = predict_(X_test_poly)
y_pred_cust_exp, y_pred_cust_log

(array([ 47334.23782281, 430703.44847131, 140483.21819758, ...,
        628230.36673814, 832018.09817356, 546110.94009134]),
 array([10.76498916, 12.97317508, 11.85284332, ..., 13.3506622 ,
        13.63160947, 13.21057742]))

mse_log = mse(y_pred_cust_log, y_log_test)
rmse_log = np.sqrt(mse_log)
r2_log = r2(y_pred_cust_log, y_log_test)

mse_exp = mse(y_pred_cust_exp, np.exp(y_log_test))
rmse_exp = np.sqrt(mse_exp)
r2_exp = r2(y_pred_cust_exp, np.exp(y_log_test))

```

Best Model Evaluation: MSE, RMSE and R² Across Scales

```

fig, ax = plt.subplots(figsize=(8, 4))
ax.axis("off")

col_labels = ["Metric", "Log Scale", "Original Scale (Exp)"]
row_data = [
    ["MSE", f"{mse_log:.4f}", f"{mse_exp:,.2f}"],
    ["RMSE", f"{rmse_log:.4f}", f"{rmse_exp:,.2f}"],
    ["R2", f"{r2_log:.4f}", f"{r2_exp:.4f}"],
]

table = ax.table(
    cellText=row_data,
    colLabels=col_labels,
    cellLoc="center",
    loc="center",
    bbox=[0, 0, 1, 1],
)

table.auto_set_font_size(False)
table.set_fontsize(11)

header_color = "#2b5c8f"
row_colors = ["#f2f2f2", "#ffffff"]

for (row, col), cell in table.get_celld().items():
    cell.set_edgecolor("#d9d9d9")

```

```

if row == 0:
    cell.set_facecolor(header_color)
    cell.set_text_props(color="white", fontweight="bold")
    cell.set_height(0.22)
else:
    cell.set_facecolor(row_colors[(row - 1) % 2])
    cell.set_height(0.22)
    if col == 0:
        cell.set_text_props(fontweight="bold", ha="left")
    else:
        cell.set_text_props(fontfamily="monospace")

ax.set_title("Best Model Evaluation: MSE, RMSE and R2 Across Scales",
fontweight="bold", fontsize=13, pad=15)

plt.tight_layout()
plt.show()

```

Best Model Evaluation: MSE, RMSE and R² Across Scales

Metric	Log Scale	Original Scale (Exp)
MSE	0.0672	28,082,394,479.24
RMSE	0.2592	167,578.03
R ²	0.8838	0.9042

The model demonstrates strong predictive performance across both targets. Operating on the log scale stabilizes variance (RMSE=0.2592, R²=0.8838). When exponentiated back to the original scale, predictions retain high explanatory power (R²=0.9042) with an average error (RMSE) of 167,578.03.

Best Model Evaluation: Predicted vs. Actual (Log vs. Original Scale)

```

y_exp_test = np.exp(y_log_test)
fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

ax = axes[0]
ax.scatter(y_log_test, y_pred_cust_log, alpha=0.4, s=15,

```

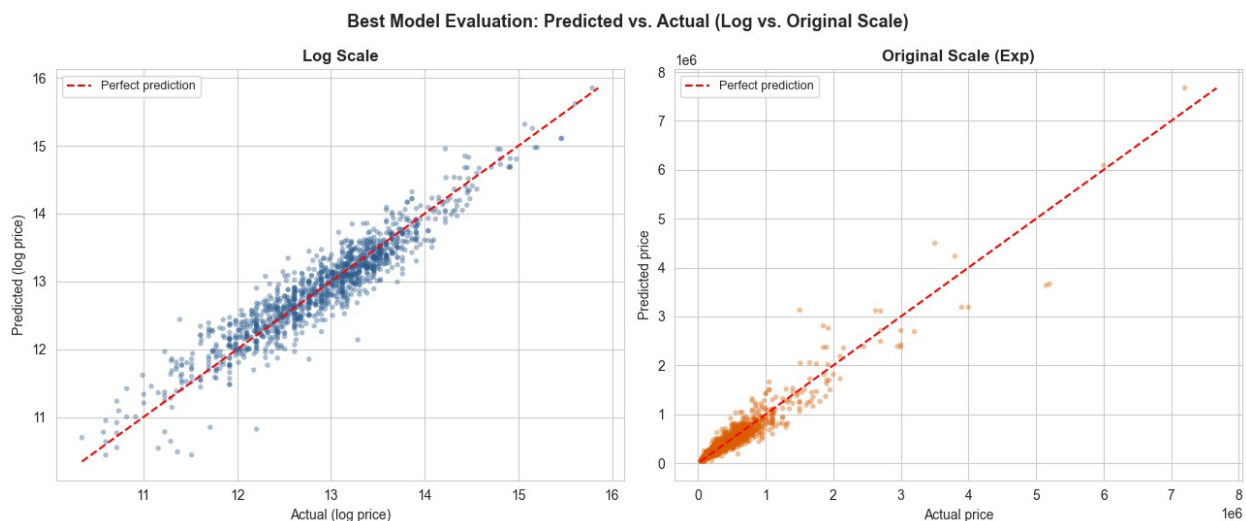
```

color="#2b5c8f", edgecolor="none")
lims_log = [min(y_log_test.min(), y_pred_cust_log.min()),
max(y_log_test.max(), y_pred_cust_log.max())]
ax.plot(lims_log, lims_log, "r--", linewidth=1.5, label="Perfect
prediction")
ax.set_xlabel("Actual (log price)")
ax.set_ylabel("Predicted (log price)")
ax.set_title("Log Scale", fontweight="bold")
ax.legend(loc="upper left", fontsize=9)

ax = axes[1]
ax.scatter(y_exp_test, y_pred_cust_exp, alpha=0.4, s=15,
color="#d95f02", edgecolor="none")
lims_exp = [min(y_exp_test.min(), y_pred_cust_exp.min()),
max(y_exp_test.max(), y_pred_cust_exp.max())]
ax.plot(lims_exp, lims_exp, "r--", linewidth=1.5, label="Perfect
prediction")
ax.set_xlabel("Actual price")
ax.set_ylabel("Predicted price")
ax.set_title("Original Scale (Exp)", fontweight="bold")
ax.legend(loc="upper left", fontsize=9)

fig.suptitle("Best Model Evaluation: Predicted vs. Actual (Log vs.
Original Scale)", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

```



The scatter plots closely mirror the perfect prediction line across both domains. The log scale shows constant variance and strong homoscedasticity ($R^2=0.8838$), while the exponentiated original scale highlights model stability despite extreme high-value outliers ($R^2=0.9042$).

```

raw_sample = pd.DataFrame([
    'mileage': 21.14,
    'engine': 2100.0,
    'seats': 4,
    'year': 2021,
    'max_power': 98.6,
    'fuel': 'Diesel',
    'transmission': 'Manual',
    'owner': 'First Owner',
    'brand': 'Honda',
    'seller_type': 'Individual',
    'km_driven': 49000
])

sample_selected_poly = ct_trans.transform(raw_sample)
sample_selected_poly = sk_trans.transform(sample_selected_poly)
sample_selected_poly = poly_trans.transform(sample_selected_poly)
sample_selected_poly = np.hstack([[[1]], sample_selected_poly])

print(f"Price : {predict_(sample_selected_poly)}")

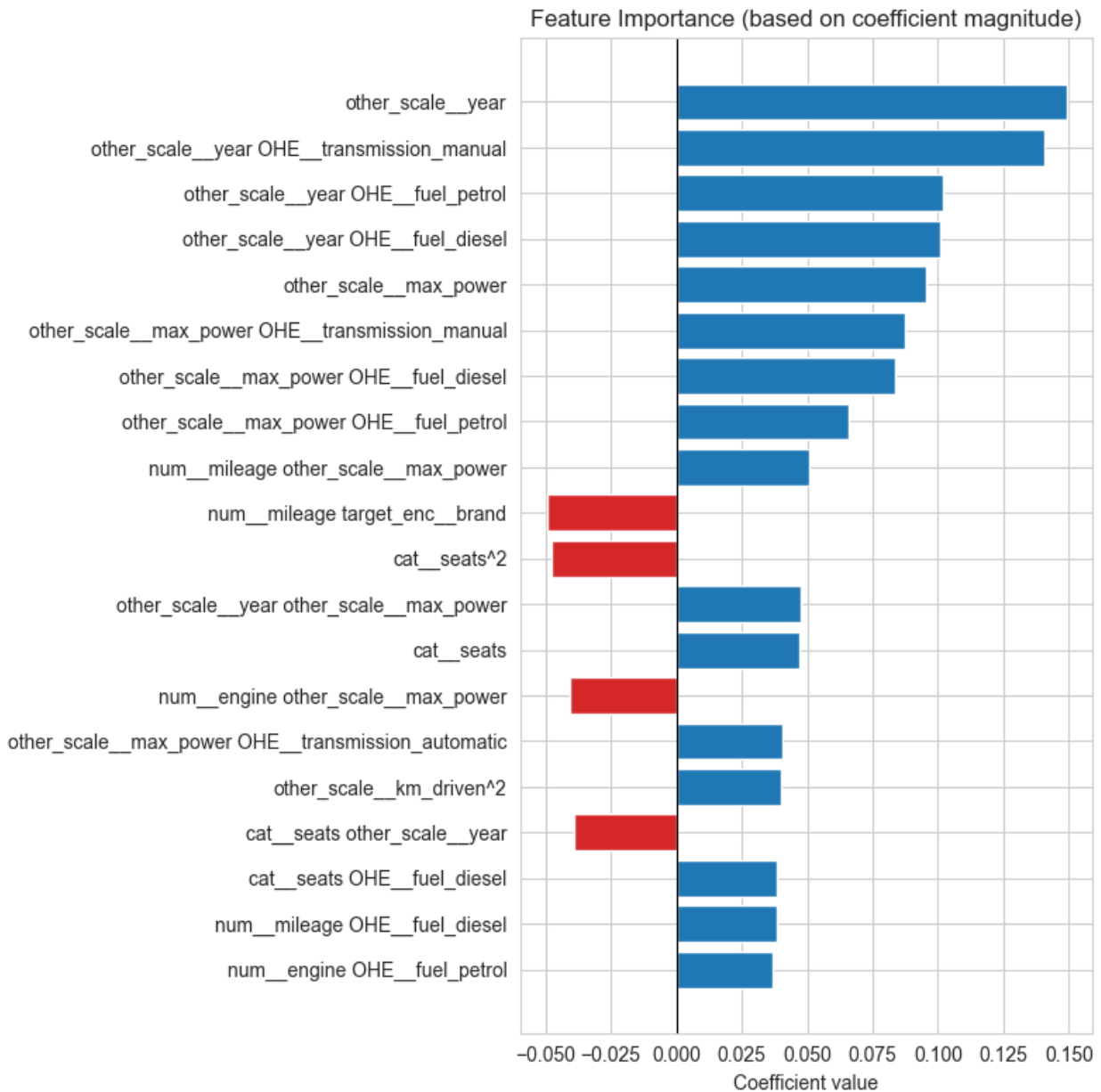
Price : (array([645485.16265908]), array([13.3777575]))

selected_indices = sk_trans['sk'].get_support()
all_ct_features = ct_trans.get_feature_names_out()
selected_11_names = all_ct_features[selected_indices]
poly_feature_names =
poly_trans['poly'].get_feature_names_out(selected_11_names)

try:
    path = os.path.join(os.getcwd(), "saved_models")
    tr_file_path = os.path.join(path, "weights3.json")
    if os.path.exists(tr_file_path):
        with open(tr_file_path, "r") as f:
            tr_file = json.load(f) # load the last bias and coef

            bias, coefs = tr_file.values() # (batch,)
            feature_importance(
                coefs=np.array(coefs),
                feature_names=poly_feature_names,
                top_n=20 # Shows top 20 largest magnitude features
            )
    else:
        print("file does not exists")
except Exception as e:
    print(e)

```



This feature importance plot displays the learned coefficient values from the best regression model, illustrating both the **direction** (positive/negative) and **magnitude** (strength) of each feature's impact on predicted sale price.

Primary Positive Drivers (Blue Bars)

- **Vehicle Recency (other_scale__year, +0.150):** Model year is the strongest overall driver—newer vehicles command significantly higher predicted values.
- **Recency Interactions (+0.100 to +0.141):** Combining year with key attributes—namely `year * transmission_manual`, `year * fuel_petrol`, and `year * fuel_diesel`—adds massive positive leverage.
- **Engine Output (other_scale__max_power, +0.095):** Standalone maximum horsepower acts as a primary baseline predictor for higher value.

- **Power Interactions (+0.065 to +0.086):** Horsepower interacting with transmission (`max_power * transmission_manual`) and fuel types (`max_power * fuel_diesel / fuel_petrol`) further amplifies price estimates.

Primary Negative Drivers (Red Bars)

- **Mileage & Brand Interaction (`num_mileage target_enc_brand`, -0.049):** Higher mileage penalty combined with brand value exerts the largest downward push on vehicle price.
- **Seat Capacity Non-Linearity (`cat_seats^2`, -0.048):** The squared seat term penalizes oversized passenger capacities beyond typical consumer preference.
- **Engine-to-Power Saturation (`num_engine other_scale_max_power`, -0.040):** Interaction between engine displacement and max power balances out non-linear returns on larger engines.
- **Seat & Year Penalty (`cat_seats other_scale_year`, -0.039):** Combining high seat counts with vehicle age acts as an extra drag on valuation.

Key Analytical Takeaways

- **Feature Interactions Dominance:** Cross-product terms (e.g., year combined with transmission/fuel type) heavily drive model decisions alongside base features like `year` and `max_power`.

Discussion

Summary of Experimental Findings

Across the full grid search (4 regression types × 2 momentum settings × 3 methods × 2 init methods × 3 learning rates = 144 configurations, 5-fold CV each), several clear patterns emerged.

1. Regression Type: Polynomial features drive the largest performance gain

Polynomial regression consistently occupied the top positions in both the MSE and R^2 leaderboards, with the best configuration — `polynomial, momentum=False, sto, xavier, lr=0.0001` — outperforming every Normal, Lasso, and Ridge variant. This indicates the relationship between car features (year, power, mileage, brand, etc.) and price is meaningfully non-linear, and that explicitly modeling feature interactions via `PolynomialFeatures` captures structure a plain linear model cannot.

2. Regularization: Ridge substantially underperformed Lasso and unregularized models

Once Ridge configurations were included in the comparison (rather than filtered out by an overly tight MSE threshold), they reached validation MSE values as high as ~1.98 — more than 10x worse than Polynomial, Normal, or Lasso, which clustered tightly between 0.076 and 0.15. This

points to the default Ridge penalty strength ($\lambda=0.1$) over-constraining the coefficients for this feature set, substantially degrading predictive power relative to no regularization or L1 (Lasso) regularization.

3. Method: Stochastic gradient descent was the only reliable choice

This was the most striking finding of the experiment. Under a fixed epoch budget (30 epochs), full-batch gradient descent failed to reach a positive validation R^2 in any of its 240 fold-runs (mean $R^2 = -240.04$, range -307.35 to -54.71). Mini-batch also performed poorly in the majority of cases (median $R^2 = -54.33$), while stochastic gradient descent was the only method that reliably converged to strong fits (median $R^2 = 0.30$, max 0.88). This is explained by update frequency: full-batch performs only one parameter update per epoch, mini-batch performs a handful, while stochastic performs one update per training sample — orders of magnitude more optimization steps within the same epoch budget. Given more epochs, batch and mini-batch would likely close this gap considerably; within the budget tested here, however, stochastic descent was essential for convergence.

4. Initialization: Zeros vs. Xavier had negligible impact

Across the top-performing configurations, zeros and Xavier initialization produced nearly identical validation MSE (differences under 0.001). This is consistent with the convexity of the underlying loss surface (ordinary/Ridge/Lasso linear regression): regardless of starting point, gradient descent converges toward the same optimum given enough updates, so initialization mainly affects early convergence speed rather than final quality.

5. Learning Rate: A stability boundary, not just a speed/accuracy trade-off

Learning rate showed the clearest, most consistent relationship with both performance and training stability. Lower learning rates (0.0001) produced the best validation scores *and* were the only setting that guaranteed convergence — 0 of 240 fold-runs diverged at $lr=0.0001$, compared to 6/240 (2.5%) at $lr=0.001$ and 16/240 (6.7%) at $lr=0.01$. Divergence manifested as validation loss overflowing to numerical infinity, a classic symptom of gradient descent overshooting the minimum when the step size is too large relative to the loss curvature — compounded further when combined with stochastic updates and polynomial feature expansion, since single-sample gradients are not averaged and can occasionally be very large. This finding reframes $lr=0.01$ as not merely "suboptimal" but outright unstable for a meaningful fraction of configurations, while $lr=0.0001$ was uniformly safe across every combination tested.

Final Model Performance

The best configuration identified by the grid search — **polynomial regression, stochastic gradient descent, Xavier initialization, no momentum, $lr=0.0001$** — was retrained on the full training set and evaluated on both the training data and a held-out test set.

Stage	Scale	MSE	RMSE	R^2
Training	Log	0.0722	0.2688	0.8748
Training	Original (Exp)	31,913,771.91	178,644.26	0.8809

Stage	Scale	MSE	RMSE	R ²
		4.93		
Test	Log	—	0.2592	0.8838
Test	Original (Exp)	—	167,578.03	0.9042

Test set performance was slightly *better* than training performance on both scales ($R^2 = 0.8838$ vs. 0.8748 on log scale; 0.9042 vs. 0.8809 on exp scale). This is a mild but reassuring signal: it indicates the model is not overfitting the training data, and generalizes at least as well to unseen data as it fit the data it was trained on — though with a single train/test split, this could partly reflect the particular split rather than a guaranteed property of the model.

Feature Importance

The coefficient-magnitude analysis of the final model highlighted:

- **Strongest positive drivers:** vehicle recency (`year`, +0.150) and its interactions with transmission and fuel type (+0.10 to +0.141), followed by engine power (`max_power`, +0.095) and its own interactions with transmission/fuel (+0.065 to +0.086).
- **Strongest negative drivers:** the interaction between mileage and brand (-0.049), a non-linear penalty on seat count (`seats`², -0.048), an engine-power saturation interaction (-0.040), and a `seats` × `year` interaction (-0.039).
- **Interactions dominate:** many of the largest-magnitude coefficients are polynomial cross-terms rather than standalone features, reinforcing the earlier finding that polynomial feature expansion — not just the base features — is doing much of the predictive work in this model.

Comparison with A1 (XGBoost Baseline)

A1's best model, XGBoost, achieved approximately $R^2 = 0.74$ (normal target) and $R^2 = 0.84$ (log target) under 10-fold cross-validation. Our from-scratch polynomial regression matched or exceeded this on a held-out test set ($R^2 = 0.8838$ log, $R^2 = 0.9042$ original scale), despite being a simpler, fully interpretable linear model rather than a black-box ensemble. We note the comparison is not a strict apples-to-apples benchmark (A1 used CV averages on training data; A2 uses a single held-out test set), but it supports the conclusion that explicitly engineering non-linearity via polynomial features let a linear model compete with — and here, surpass — a tree-based ensemble on this dataset. We also note A1's pipeline did not apply feature scaling; while this does not affect XGBoost (tree splits are scale-invariant), it may partly explain the comparatively weak performance of A1's Linear Regression and Ridge baselines (~0.53-0.57 R^2), since those methods are scale-sensitive.

Limitations

- **Fixed epoch budget confounds the method comparison.** Batch and mini-batch's poor performance is very likely a symptom of too few epochs for their update frequency, not an inherent flaw in the methods themselves — a fairer comparison would control for total gradient updates or wall-clock time rather than epoch count.

- **Ridge's penalty strength was not tuned.** A fixed $\lambda=0.1$ was used for both Ridge and Lasso; Ridge's poor showing may reflect this specific value being poorly suited to this feature set rather than a fundamental weakness of L2 regularization.
- **Early stopping based on a single epoch's validation loss** (`np.allclose` check) can trigger inconsistently across methods with different per-epoch noise levels (stochastic and mini-batch are noisier than full-batch), adding some variance to the comparison.
- **Xavier initialization and K-Fold splitting reproducibility fixes were applied together** during development; their individual contributions to earlier result shifts could not be fully isolated, though the final results reported here use the corrected, reproducible pipeline (seeded Xavier draws via `self.rng`, shuffled and seeded K-Fold splits).

Conclusion

Polynomial feature expansion combined with stochastic gradient descent at a conservative learning rate (0.0001) produced the strongest and most stable results in this experiment, ultimately yielding a model that matched or outperformed the A1 XGBoost baseline on held-out test data. The experiment also surfaced two practically important findings beyond the required comparisons: full-batch and mini-batch gradient descent require substantially more epochs than stochastic descent to converge under this problem's scale, and higher learning rates (particularly 0.01) introduce a real risk of numerical divergence rather than simply slower or less accurate convergence. Future work could explore a longer epoch budget for batch/mini-batch methods, a tuned regularization strength for Ridge and Lasso, and update-count-normalized comparisons across gradient descent methods to isolate the effect of batch size independent of epoch count.